

Integração e Entrega Contínua — DevOps

Semana 01: Cultura DevOps, Filosofia CAMS e Modelo de Ramificação GitFlow

Unidade Curricular: DevOps (40h)

Planta Didática Smart N1 | Ecossistema Integrado de Aprendizagem

1. O Dilema Tradicional: Dev vs Ops



Desenvolvimento (Dev)

- Objetivo: Entregar novas funcionalidades rapidamente.
- Incentivo: Mudança contínua e inovação.
- Visão de Sucesso: Código escrito e enviado para teste.



Operações (Ops)

- Objetivo: Manter o sistema estável e no ar.
- Incentivo: Estabilidade e aversão ao risco.
- Visão de Sucesso: Zero paradas não planejadas em produção.

2. O que é DevOps?

- União de Pessoas, Processos e Tecnologia: Movimento cultural para entregar valor contínuo aos usuários.
- Quebra de Silos: Fim da barreira competitiva entre desenvolvedores e administradores de sistemas.
- Responsabilidade Compartilhada: Quem constrói a aplicação também é responsável por sua operação (*You build it, you run it*).
- Foco na Velocidade e Qualidade: Acelerar o ciclo de vida do software reduzindo drasticamente falhas em produção.

3. A Filosofia CAMS (Os 4 Pilares DevOps)

Slide 04/17

C - Culture (Cultura)

Comunicação aberta, confiança mútua e responsabilidade compartilhada.

A - Automation (Automação)

Eliminação de tarefas manuais repetitivas via esteiras CI/CD.

M - Measurement (Mensuração)

Monitoramento constante de métricas de software e infraestrutura.

S - Sharing (Compartilhamento)

Disseminação transparente de ferramentas, conquistas e lições aprendidas.

Empatia e Transparência

Construir um ambiente sem culpa (blameless culture) em incidentes.

Melhoria Contínua

Ciclos iterativos rápidos de aprendizado e adaptação.

4. Os Três Caminhos do DevOps (The DevOps Handbook)

- Primeiro Caminho (Fluxo): Acelerar o fluxo de valor da esquerda para a direita (Dev → Ops → Cliente).
- Segundo Caminho (Feedback): Criar ciclos rápidos e constantes de feedback da direita para a esquerda.
- Terceiro Caminho (Aprendizado): Cultivar uma cultura de experimentação contínua e maestria técnica.

5. O Ciclo de Vida do Software em DevOps

Slide 06/17

1. Plan (Planejar)

Definição de tarefas, histórias de usuário e backlog (Jira/GitHub).

2. Code (Codificar)

Desenvolvimento do código fonte e versionamento Git.

3. Build (Compilar)

Empacotamento de artefatos e imagens Docker.

4. Test (Testar)

Execução automatizada de suítes de testes de unidade e integração.

5. Release & Deploy

Publicação automatizada em homologação e produção.

6. Operate & Monitor

Monitoramento contínuo de métricas DORA e logs de produção.

6. Versionamento de Código com Git

- Sistema de Controle de Versão Distribuído (DVCS): Cada desenvolvedor possui uma cópia completa do histórico do projeto.
- Snapshots de Código: O Git registra alterações como um grafo direcionado acíclico (DAG) de commits.
- Benefícios: Rastreabilidade total, facilidade de reverter bugs e suporte ao trabalho simultâneo em equipe.
- Repositórios Remotos: GitHub, GitLab e Bitbucket atuando como servidores centrais de colaboração.

7. O Modelo de Ramificação GitFlow

- O que é o GitFlow? Modelo de gerenciamento de branches criado por Vincent Driessen para organizar lançamentos de software.
- Isolamento de Código: Garante que novas funcionalidades não quebrem o ambiente de produção estável.
- Padronização de Branches: Define nomes e papéis específicos para cada ramificação no repositório.
- Suporte ao Desenvolvimento Paralelo: Permite que múltiplos desenvolvedores trabalhem simultaneamente.

8. Branches Principais: main e develop

Slide 09/17

Branch `main` (Produção)

- Contém estritamente código estável em produção.
- Cada commit em `main` corresponde a uma nova versão lançada.
- Protegida contra commits diretos (requer Pull Request).

Branch `develop` (Integração)

- Branch de integração contínua do desenvolvimento.
- Reflete as últimas alterações prontas para a próxima versão.
- Base para a criação de novas funcionalidades (`feature/\*`).

9. Branches de Apoio no GitFlow

feature/*

Ramificada a partir de `develop`. Usada para desenvolver uma nova funcionalidade.

release/*

Ramificada a partir de `develop`. Usada para preparar a publicação de uma nova versão.

hotfix/*

Ramificada a partir de `main`. Usada para corrigir emergências críticas em produção.

Merge de Feature

Após aprovada em Code Review, é mesclada de volta na branch `develop`.

Merge de Hotfix

Após corrigida, é mesclada simultaneamente em `main` e em `develop`.

Versionamento

Uso de tags semânticas (ex: `v1.0.0`) em cada merge na `main`.

10. Fluxo de Trabalho Prático no GitFlow

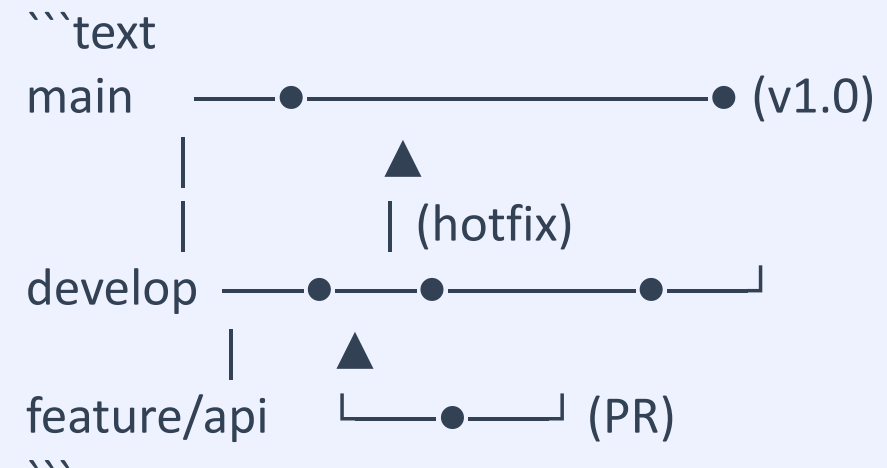
Slide 11/17

Passo a Passo de Desenvolvimento

1. Alternar para a branch de desenvolvimento:
``git checkout develop``
2. Criar uma branch de funcionalidade:
``git checkout -b feature/setup-api``
3. Desenvolver o código e realizar commits locais.
4. Enviar a branch para o repositório remoto:
``git push origin feature/setup-api``
5. Abrir Pull Request para a branch ``develop``.



Exemplo de Árvore de Branches



11. Boas Práticas: Commits Semânticos (Conventional Commits)

- Padronização de Mensagens: Facilita a leitura do histórico de commits e a automação de changelogs.
- ``feat:`` Adição de uma nova funcionalidade ao sistema.
- ``fix:`` Correção de um bug ou comportamento incorreto.
- ``docs:`` Alterações na documentação (ex: README.md).
- ``refactor:`` Reformulação de código sem alterar sua funcionalidade externa.

12. Introdução à Integração Contínua (CI)

- O que é CI (Continuous Integration)? Prática de automatizar a compilação e teste do código a cada push no Git.
- Detecção Precoce de Bugs: Erros são identificados minutos após o commit, reduzindo o custo de correção.
- Ferramentas Populares: GitHub Actions, GitLab CI/CD, Jenkins, CircleCI.
- Status da Esteira: Feedback visual com status verde (*Success*) ou vermelho (*Failed*).

13. O DevOps no Ecosystema Smart N1

Slide 14/17

- Papel no Semestre: Empacotar a plataforma FactoryHub em contêineres Docker e automatizar os deploys.
- Versão Semanal: Evolução desde a configuração de branches do GitFlow até esteiras de CI/CD corporativas.
- Integração com o Frontend: Conexão do repositório GitHub com a plataforma Vercel para *Preview Deployments*.
- Qualidade de Código: Execução automatizada de testes com `pytest` a cada Pull Request.

14. Desafio Prático da Semana 01 (DevOps)

Slide 15/17

- Tarefa 1: Inicializar o repositório Git local da equipe e configurar as identidades (``git config``).
- Tarefa 2: Criar as branches base do GitFlow (``main`` e ``develop``).
- Tarefa 3: Criar uma branch ``feature/setup-inicial``, realizar o primeiro commit semântico e enviar ao GitHub.
- Entregável: Repositório configurado no GitHub com as branches do GitFlow e commit inicial.

15. Referências Bibliográficas Oficiais

- KIM, Gene; HUMBLE, Jez; DEBOIS, Patrick; WILLIS, John. The DevOps Handbook. IT Revolution Press, 2016.
- HUMBLE, Jez; FARLEY, David. Entrega Contínua: Como Entregar Software de Forma Rápida e Confiável. Bookman, 2014.
- CHACON, Scott; STRAUB, Ben. Pro Git. 2ª ed. Apress, 2014. (Disponível em: git-scm.com/book).
- DRIESSEN, Vincent. A successful Git branching model. Disponível em: nvie.com/posts/a-successful-git-branching-model.

Conclusão e Próximos Passos

DevOps é a ponte que une velocidade de desenvolvimento e estabilidade operacional.

Próxima Aula: Preview Deployments na Vercel e conexão com repositórios remotas!
Dúvidas e Discussão?